FOR VIDEO EDITORS · FREE PROMPT PACK

# 10 Fable 5 Prompts Editors Use to Cut Twice as Fast.

Footage triage, batch renders, revision decoding, delivery QC — 10 copy-paste master prompts built for the edit bay. Each one produces a **reusable system**, not a one-off answer.

FABLE 5 FREE WINDOW CLOSES JULY 19

Fable 5 is included in paid Claude plans until July 19, 2026, 11:59pm PT — after that it moves to usage credits. Run these while it's included.

FOOTAGE TRIAGE · BATCH PIPELINES · REVISION DECODE · DELIVERY QC

By Waseem Nasir · SkynetLabs · @skynetlabs

# Three steps. That's it.

No plugins, no setup. If you have a paid Claude plan, you already have everything you need.

**1**

### Open Claude and pick Fable 5

Go to `claude.ai` (or Claude Code in your terminal) and select **Fable 5** from the model picker. It's included in every paid plan until July 19, 2026 — up to 50% of your weekly usage.

**2**

### Paste a prompt, fill the brackets

Copy any prompt from this pack and replace everything in `[BRACKETS]` with your real details — your footage log, your client's revision email, your actual delivery specs. The more real material you paste, the better the output. Fable 5 handles huge context, so don't hold back.

**3**

### Save the output as a system

Every prompt here returns something reusable — a checklist, a template, a script. Save it. When a prompt earns its keep twice, ask Claude: `"Convert what we just did into a reusable skill"` — and it's yours on any model, forever.

> **Why these work:** each prompt follows the same skeleton — role, goal, real context, hard constraints, exact output format. Fable 5 plans backward from goals; these prompts hand it the goal and get out of the way.

## 01 The Footage Triage System

**WHEN TO USE** — you're staring at a 200-clip card dump and don't know where the story is.

```
You are a senior editor who logs and selects footage for a
living. GOAL: Build my footage triage system for this project — a
ranked selects list plus the reusable triage checklist I run on
every future dump. CONTEXT: The project:
[REEL / WEDDING / DOC / BRAND FILM + TARGET LENGTH]. My clip log or file list:
[PASTE FILENAMES + TIMESTAMPS + ANY NOTES].
The story or emotion the final cut must land:
[ONE SENTENCE]. Delivery format:
[9:16 / 16:9, PLATFORM]. CONSTRAINTS: Rank
clips into HERO / SUPPORT / CUT — every HERO pick needs a one-line
reason tied to the story goal. Flag coverage gaps I need pickups
for. No clip earns HERO for being pretty; it earns it for doing
narrative work. PROCESS: Map the story beats first, then assign
clips to beats, then rank inside each beat. OUTPUT FORMAT: Beats
table, HERO/SUPPORT/CUT lists with reasons, coverage-gap list, and
the blank TRIAGE.md checklist for next time.
```

**PRO TIP** — paste your raw filenames even if they're ugly. DJI_0417.MP4 plus your one-word note is enough for it to build the beat map.

## 02 The FFmpeg Batch Pipeline

**WHEN TO USE** — you keep doing the same export dance by hand: crop, grade, captions, loudness, five aspect ratios.

```
You are a video pipeline engineer who automates repetitive render
work with ffmpeg. GOAL: Build my batch render pipeline as a
runnable script — one command turns a folder of masters into every
deliverable variant I ship. CONTEXT: My OS and shell:
[WINDOWS/MAC + POWERSHELL/BASH]. Input
footage specs:
[CODEC, RESOLUTION, BIT DEPTH — e.g. 10-BIT HEVC FROM
  DJI]. Deliverables I produce every time:
[LIST: 9:16 CROP, 1:1, BURNED CAPTIONS, -14 LUFS AUDIO,
  THUMBNAIL FRAME…]. My GPU: [NVIDIA MODEL OR NONE].
CONSTRAINTS: Use GPU encoding (NVENC) when available with a CPU
fallback. Every output yuv420p + faststart so platforms accept it.
Loudness normalized. The script must skip already-rendered files
so re-runs are safe. Comment every non-obvious flag. PROCESS:
Confirm the variant matrix in a table first, then write the
script, then walk one file through every branch. OUTPUT FORMAT:
Variant matrix, the complete script in one block, and a GOTCHAS
section for my specific codec.
```

**PRO TIP** — tell it your exact camera. 10-bit HEVC sources have pixel-format traps that generic commands silently mangle.

## 03 The Caption Style Codex

**WHEN TO USE** — your captions look different on every project and you re-decide fonts, colors, and safe zones from scratch each time.

```
You are a motion-design systems lead who locks brand styles into
              reusable specs. GOAL: Write my caption style codex — one locked
              spec (fonts, colors, sizes, positions, animation rules) I hand to
              any tool or collaborator, so every video looks like the same brand
              cut it. CONTEXT: My brand or my client's brand:
              [COLORS AS HEX, FONTS, LOGO RULES].
              Platforms I deliver to: [LIST]. Screenshots
              or links of caption looks I like:
              [DESCRIBE OR PASTE]. My tools:
              [PREMIERE / CAPCUT / FFMPEG+ASS / DAVINCI].
              CONSTRAINTS: Respect platform safe zones — top and bottom UI bars
              and the right-side action column eat text. Every choice gets a
              stated reason. Include a render-safe font fallback for tools that
              can't load custom fonts. Define the ONE accent treatment for
              emphasized words — not five. PROCESS: Audit the pasted references
              for what's actually consistent, then write the codex, then produce
              the quick-reference card version. OUTPUT FORMAT: The full codex
              (tokens, rules, do/never lists) plus a 10-line cheat-sheet I can
              pin.
```

**PRO TIP** — ask it to output the spec in your tool's native terms too: ASS style lines for ffmpeg, or an Essential Graphics checklist for Premiere.

## 04 The Client Revision Decoder

**WHEN TO USE** — the client sent "make it pop, also my wife thinks the middle drags" and you need an actual task list.

```
You are a post-production supervisor who translates client
              feedback into edit decisions. GOAL: Convert this revision note
              into a numbered, unambiguous edit task list — plus flag anything
              that needs a clarifying question BEFORE I touch the timeline.
              CONTEXT: The client's feedback, verbatim:
              [PASTE THE EMAIL / VOICE-NOTE TRANSCRIPT].
              What was agreed in the original scope:
              [DELIVERABLES + REVISION ROUNDS INCLUDED].
              Timecoded structure of the current cut:
              [ROUGH SECTION MAP]. CONSTRAINTS: Separate
              what they SAID from what they likely MEAN — flag every
              interpretation you made. Mark each task as IN-SCOPE or SCOPE-CREEP
              against the agreement. Vague feedback becomes a clarifying
              question, never a guess. Estimate minutes per task. PROCESS: Parse
              the note line by line, map each fragment to a timeline region,
              then compile. OUTPUT FORMAT: Task table (timecode, task,
              interpretation flag, scope flag, est. minutes), the
              clarifying-questions message ready to send, and a one-line summary
              of total revision time.
```

**PRO TIP** — the SCOPE-CREEP column is your paycheck protector. Paste your actual contract language into CONTEXT for sharper calls.

## 05 The Hook Doctor

**WHEN TO USE** — before you export anything short-form. The first three seconds decide everything; stop grading your own hook.

```
You are a short-form retention analyst. You judge opening seconds
coldly, like a viewer with a thumb on the screen. You default to
FAIL on uncertainty. GOAL: Grade the hook of my cut and give me
ranked fixes — so I stop shipping openers only their editor loves.
CONTEXT: Frame-by-frame description of my first 5 seconds:
[WHAT'S ON SCREEN, TEXT SHOWN, FIRST SPOKEN LINE, SOUND]. Platform: [TIKTOK / REELS / SHORTS]. Who
must stop scrolling: [AUDIENCE]. The
video's promise:
[WHAT THEY GET IF THEY STAY]. CONSTRAINTS:
Judge on checkable criteria: is there motion or change in second
one, is the promise visible without sound, does the text card
duplicate the audio or add to it, would a stranger know what this
video is about by second three. Reasons before scores. No "it
depends." PROCESS: Score each criterion, state the single biggest
leak, then propose 3 alternative hook structures using ONLY
footage I already have. OUTPUT FORMAT: Scorecard, verdict
PASS/FAIL, the leak, 3 rewrite options ranked by effort.
```

**PRO TIP** — describe honestly. If your first second is a wide establishing shot, say so, and let it hurt.

## 06 The Repurposing Matrix

**WHEN TO USE** — you cut one long video and it dies as one upload. One master should feed weeks of shorts.

```
You are a content repurposing strategist for video-first creators
and studios. GOAL: Build the repurposing matrix for my long-form
video — every clip-worthy moment mapped to a platform, format, and
hook — plus the reusable extraction rules for every future master.
CONTEXT: The transcript or detailed outline:
[PASTE — FABLE 5 HANDLES THE FULL TRANSCRIPT]. Runtime: [LENGTH]. Platforms and my
per-week capacity: [HONEST NUMBER]. Which
moments got a reaction when I cut similar videos:
[ANY PAST SIGNAL]. CONSTRAINTS: Each
extracted clip must stand alone — no "as I said earlier"
cold-opens. Every clip gets its own hook line, not the master's
title. Cap the list at my stated capacity; rank the rest as bench.
No clip over 60 seconds unless the moment demands it. PROCESS:
Sweep the transcript for self-contained payoffs, rank by hook
strength, assign platforms, then write the extraction rules.
OUTPUT FORMAT: Clip table (timecode in/out, hook line, platform,
format note), bench list, and the RULES.md for next month's
master.
```

**PRO TIP** — run this the day you deliver the long cut, while export settings are still warm. The matrix doubles as your upsell to the client.

## 07 The Silent-Failure Render Retrofit

**WHEN TO USE** — your overnight batch render died at clip 3 of 40 and you found out at 9am.

```
You are a reliability engineer for media pipelines. GOAL: Audit
              my render workflow for every way it fails silently, and produce
              the retrofit — logging, checks, and notifications — so a dead
              render wakes me up instead of wasting the night. CONTEXT: My
              current pipeline:
              [PASTE THE SCRIPT / DESCRIBE THE WATCH-FOLDER SETUP / NLE
                RENDER QUEUE]. What a failed delivery costs me:
              [STAKES]. How I currently discover
              failures:
              [HONEST — USUALLY "I CHECK THE FOLDER IN THE MORNING"]. CONSTRAINTS: Every failure mode gets detection +
notification +
              a retry-or-skip decision. Verify outputs, not exit codes — a
              zero-byte MP4 with exit 0 is still a failure; check duration and
              stream count per file. No failure may terminate the whole batch.
              Include a post-batch summary report. PROCESS: Trace each stage's
              failure modes, map what I'd currently see (nothing), then design
              fixes in blast-radius order. OUTPUT FORMAT: Failure-mode table
              (stage, what breaks, current behavior, fix), the retrofitted
              script or checklist, and my reusable RENDER-STANDARD.md.
```

**PRO TIP** — the output-verification step catches the worst class of failure: renders that "succeed" but are broken. Probe every file, trust none.

## 08 The Project Handoff Brief

**WHEN TO USE** — you're bringing in a second editor, or future-you will reopen this project in 6 months and remember nothing.

```
You are a post-production supervisor who onboards editors onto
              in-progress projects. GOAL: Write the handoff brief for my project
              — the one document that lets another editor (or future me) open it
              cold and continue without breaking anything. CONTEXT: NLE and
              version:
              [PREMIERE / RESOLVE / FCP + VERSION].
              Project structure:
              [BIN LAYOUT, SEQUENCE NAMING, PROXY SETUP].
              Where media lives:
              [DRIVES / PATHS / CLOUD]. Client quirks and
              past mistakes on this account:
              [LIST HONESTLY]. Delivery specs:
              [EXACT]. CONSTRAINTS: Only facts that stay
              true — no session state. Include a NEVER-DO section built from the
              past mistakes. Every path exact and every naming convention
              spelled out. Under 120 lines. PROCESS: Draft it, then audit: would
              a fresh editor with only this doc make the mistakes I listed?
              Patch until no. OUTPUT FORMAT: One markdown brief — Overview,
              Structure, Media Map, Conventions, Delivery Specs, Never Do.
```

**PRO TIP** — the NEVER-DO section is the whole value. "Client rejects any cut using the drone footage from day 2" saves a full revision round.

## 09 The Pre-Delivery QC Gate

**WHEN TO USE** — before every export leaves your machine. You've shipped a wrong-aspect-ratio file once. Never again.

```
You are a broadcast QC specialist. You are skeptical by default
              and you check files, not vibes. GOAL: Build my pre-delivery QC
              gate — the checklist and verification commands I run on every
              export before it reaches a client or a platform. CONTEXT: What I
              deliver: [FORMATS + PLATFORMS]. Past
              delivery failures that actually happened:
              [LIST — WRONG CROP, DEAD AUDIO CHANNEL, COPYRIGHT FLAG, TYPO IN
                LOWER THIRD…]. My tools: [FFPROBE AVAILABLE? OS?].
              CONSTRAINTS: Past failures become checklist items first. Every
              technical check gets a runnable command (probe duration, streams,
              loudness, pixel format) — not "check the audio." Include the
              human-eye pass: title-card typos, safe-zone violations,
              first-frame-as-thumbnail check, and a music-rights confirmation
              line. MUST-PASS items separated from warnings. PROCESS: Convert my
              failures to checks, add the standard technical sweep, order by
              cost-of-miss. OUTPUT FORMAT: The QC-GATE.md checklist with
              commands inline, split into MUST-PASS / WARN.
```

**PRO TIP** — add "play the last 10 seconds" as a MUST-PASS. Truncated endings survive every automated check and die in public.

## 10 The Editing Package Builder

**WHEN TO USE** — every job is a custom quote and your margin depends on estimating at midnight. Package it.

```
You are a pricing strategist for freelance video editors and
              small studios. GOAL: Design my productized editing packages — 2-3
              fixed-scope offers built from work I've actually delivered — with
              pricing, scope boundaries, and the exact language to present them.
              CONTEXT: Real projects I've delivered and roughly what each took:
              [LIST: TYPE, HOURS, WHAT I CHARGED]. How I
              quote today: [HONEST]. Objections I hear:
              [REAL ONES]. My minimum viable rate:
              [FLOOR]. CONSTRAINTS: Every package lists
              what's NOT included — revision rounds capped and numbered,
              source-file handoff priced separately, rush fees defined. Middle
              package = the obvious choice. Frame value as client outcomes
              (videos shipped per month, turnaround), never as my hours. Include
              the upgrade path between tiers. PROCESS: Cluster my past projects
              into natural packages, price against value, stress-test against my
              listed objections. OUTPUT FORMAT: Package table, per-package
              one-paragraph pitch, NOT-included lists, objection responses, and
              rush/revision rules.
```

**PRO TIP** — "unlimited revisions" is how editing businesses die. Let the NOT-included list do the negotiating before the client does.

WANT IT DONE FOR YOU?

# We build the systems.

# You keep shipping cuts.

SkynetLabs builds AI-powered pipelines for editors, creators, and studios — batch render systems, caption engines, repurposing workflows, full publishing automation. Real systems, running on real projects, published openly.

| | |
|---|---|
| WEBSITE | **skynetjoe.com** |
| SOCIAL | **@skynetlabs** |
| ALL 40 PROMPTS | **skynetjoe.com/blog/claude-fable-5-prompts-40-master-prompts** |

This pack adapts 10 of the 40 master prompts from our full Fable 5 guide — re-built for the edit bay. Get the other 30 at the link above.

© 2026 SkynetLabs · Free to share, not to resell